Практическая работа № 3 (19.10)

Тестирование, контейнеризация и CI/CD для FastAPI-приложения

Цель работы: научиться писать автоматические тесты для FastAPI-приложения, упаковывать его в Docker-образ и настраивать конвейер CI/CD в GitHub Actions с запуском тестов и публикацией образа.

Исходные данные: репозиторий из практической работы № 2.

1. Постановка задачи

1) Покрыть API автоматическими тестами (не менее 8), которые работают с реальной PostgreSQL.
2) Написать Dockerfile и полный docker-compose.yml (приложение + БД + миграции).
3) Настроить GitHub Actions:
    - на каждый push и pull request запускаются линтер и тесты;
    - при push в main после успешных тестов образ собирается и публикуется в GitHub Container Registry, а также добавляется в реестр Docker Hub.

2. Порядок выполнения

Шаг 1. Зависимости для разработки. Создайте файл requirements-dev.txt:

```
-r requirements.txt
pytest
pytest-asyncio
httpx
ruff
```

Шаг 2. Конфигурация pytest. Создайте pytest.ini:

```
ini
[pytest]
asyncio_mode = auto
asyncio_default_fixture_loop_scope = session
asyncio_default_test_loop_scope = session
testpaths = tests
```

Шаг 3. Тестовые фикстуры. Для тестов нужна отдельная БД (tasks_test). Каждый тест выполняется на чистых таблицах. Зависимость get_session подменяется через app.dependency_overrides.

Шаг 4. Тесты. Обязательные тесты:

1. создание пользователя (201);
2. повторный email (409);
3. невалидный email (422);
4. создание задачи (201);
5. создание задачи с несуществующим владельцем (404);
6. фильтр done и пагинация;
7. PATCH меняет только переданные поля;
8. DELETE, затем GET возвращает 404;
9. наличие заголовка X-Process-Time.

Шаг 5. Dockerfile. Используйте многоэтапную сборку и запуск от непривилегированного пользователя.

Шаг 6. docker-compose. Сервис app должен ждать готовности БД (healthcheck) и применять миграции перед запуском.

Шаг 7. CI. Создайте .github/workflows/ci.yml с job-ами lint, test и build.

Шаг 8. Проверка. Создайте pull request с намеренно сломанным тестом и убедитесь, что pipeline «красный». Затем исправьте тест и выполните merge.

3. Требования к отчёту

В README.md должны быть:

1. Бейдж статуса CI (![CI](https://github.com/<user>/<repo>/actions/workflows/ci.yml/badge.svg)).
2. Команды для запуска тестов локально и через Docker Compose.
3. Скриншот успешного pipeline и скриншот «упавшего» pipeline из шага 8.
4. Ссылка на опубликованный образ в GHCR.
5. Ответы на контрольные вопросы.

4. Контрольные вопросы

1. Чем CI отличается от CD (continuous delivery и continuous deployment)?
2. Как работает dependency_overrides и зачем он нужен в тестах?
3. Зачем нужна многоэтапная сборка образа?
4. Почему контейнер не следует запускать от root?
5. Как в GitHub Actions передаются секреты и почему их нельзя хранить в репозитории?
6. Что произойдёт с job-ом build, если упадёт test? Где это задаётся?
7. Где лучше применять миграции: при старте контейнера или отдельным шагом в CD? Каковы плюсы и минусы каждого варианта?